

Listas Lineares em Alocação Encadeada

Estruturas de Dados e Algoritmos

Prof. Sérgio Gorender

- Algoritmos que implementam listas com alocação sequencial são em geral simples, mas com um desempenho fraco. Em especial, devido à alocação prévia da memória, apresentam um tamanho máximo para a quantidade de elementos nas listas, e, caso seja feita uma alocação de grande quantidade de memória para uma lista pequena, a memória estará sendo utilizada de forma pouco eficiente.
- Também devemos levar em conta a maior dificuldade em se implementar mais de uma lista no mesmo arranjo.

- Com alocação sequencial, operações que envolvam várias listas, como juntar listas (merge), separar listas, são bastante complicadas de implementar e pouco eficientes.
- Utilizar alocação encadeada (ou alocação dinâmica) é mais conveniente, embora seja necessário um campo a mais em cada elemento da lista, para o ponteiro.

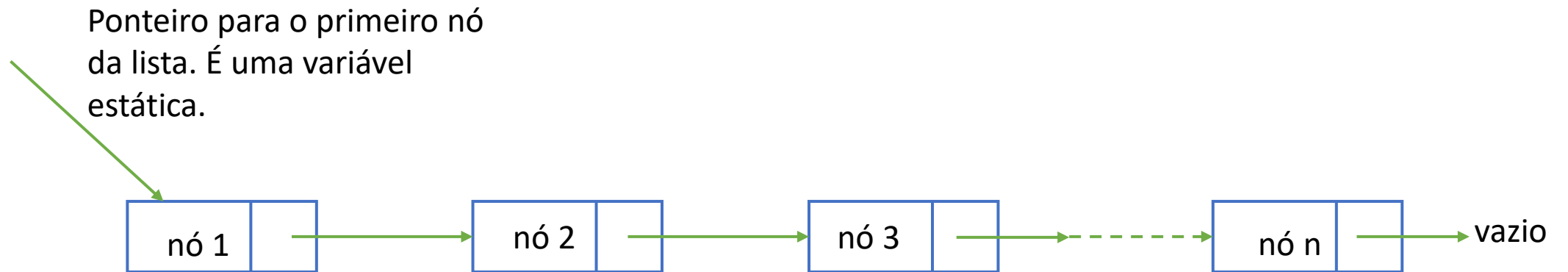
- Na alocação encadeada os nós de uma lista podem estar espalhados na memória, e não em posições contíguas. A cada novo nó inserido na lista, uma nova variável nó é dinamicamente criada, alocando memória para este nó onde for disponível.
- Para construir a sequência de nós na lista, cada nó precisa ter um campo a mais, para funcionar como ponteiro para o próximo nó.
- A alocação de memória é feita apenas quando necessário. Não há desperdício de memória. O limite para o tamanho da lista seria o da memória disponível.

Estrutura básica de um nó:

- Chave;
- Ponteiro para o próximo elemento



pt = ponteiro para o próximo elemento

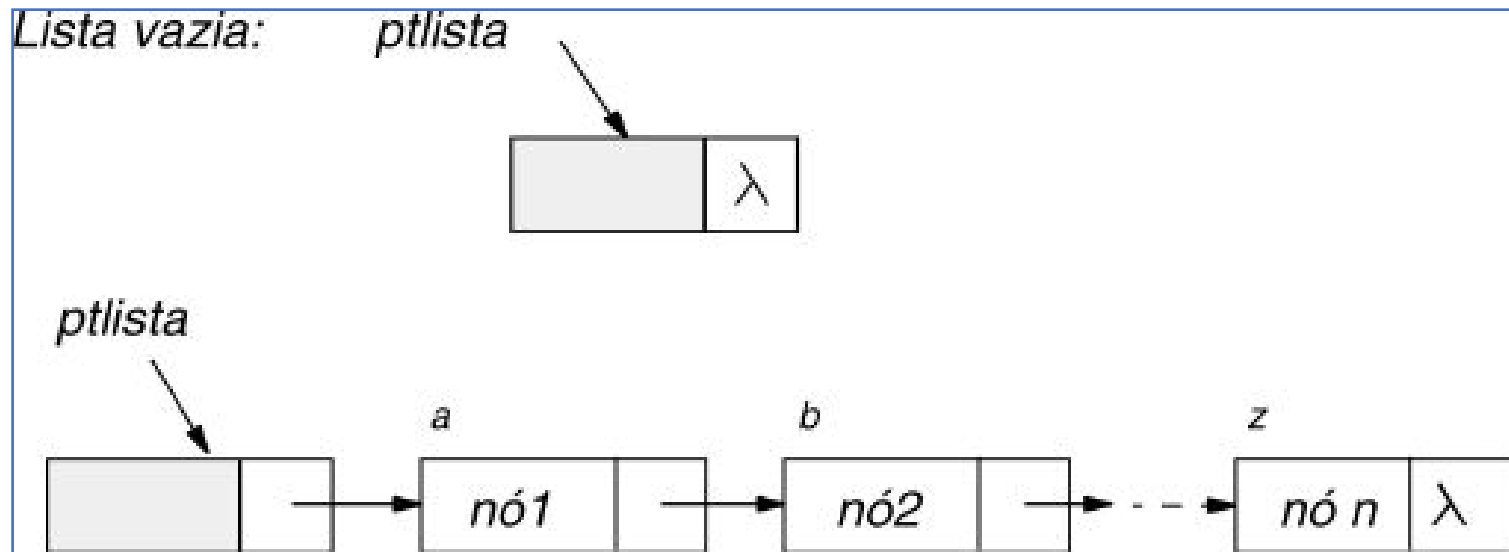


Alocação com variáveis dinâmicas

- Variáveis dinâmicas são criadas durante a execução do programa, alocando memória de forma dinâmica. Também podem ser excluídas, devolvendo a memória alocada.
- A própria linguagem faz a gestão da memória, e a alocação de memória ou sua disponibilização são feitas usando funções.

Lista Encadeada com Nó *Header*.

- O nó *Header* serve para otimizar os algoritmos, não sendo necessários testes especiais relativos à variável *ptlista*.
- A única variável estática da lista é *ptlista*, ponteiro para o nó *Header*.



Alocação com variáveis dinâmicas

- Em linguagem de algoritmo vamos adotar `new(pt)` e `free(pt)`. A função `new(pt)` retorna na variável `pt` o endereço da variável criada. A função `free(pt)` desaloca a memória ocupada pela variável apontada por `pt`.
- Em Pascal são as funções `new(pt)` e `dispose(pt)`.
- Em C temos `malloc` para ocupar e `free(pt)` para desocupar
`novo = (ListaNo*) malloc(sizeof(ListaNo)).`
- Em Python utilizamos objetos.


```
class Elemento:  
    def _init_(self):  
        self.valor = 0  
        self.prox = 0
```

```
primeiro = Elemento()
```

```
valor = int(input("Digite um valor:"))  
novo = Elemento()  
novo.valor = valor  
novo.prox = 0  
primeiro.prox = novo
```

Implementação de
Lista Encadeada em
Python, com nó
cabeçalho.

Busca em lista ordenada

Complexidade $O(n)$

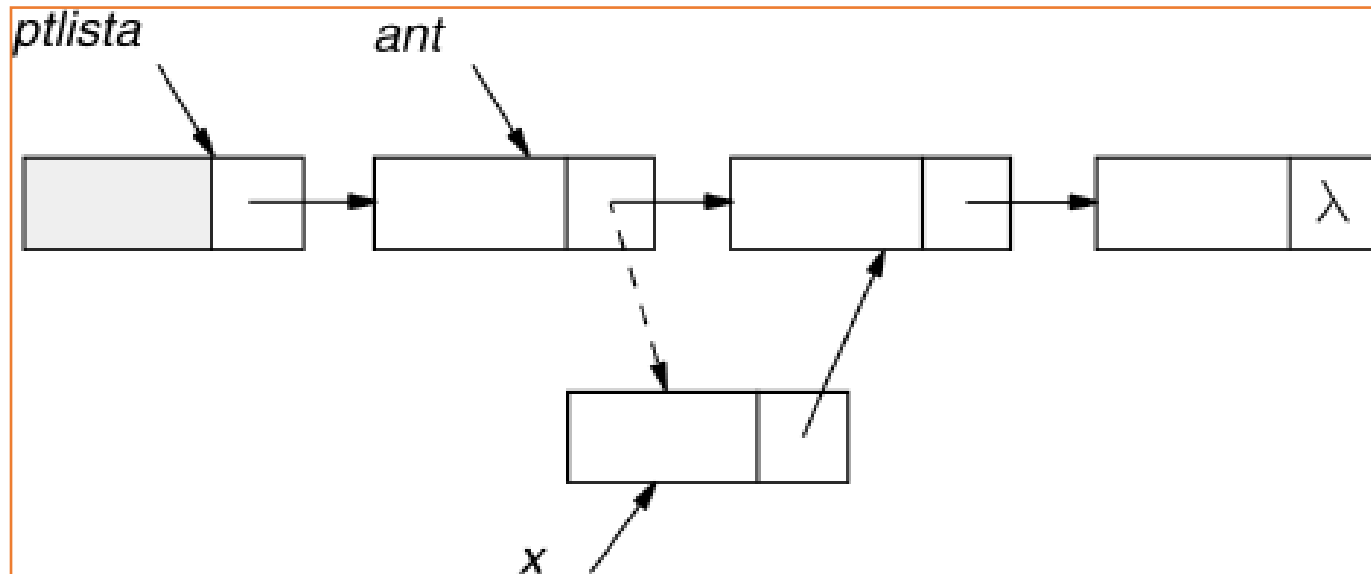
```
function busca-enc(x, ptlista)
    ant = ptlista; %anterior ao nó procurado-inicia com nó header%
    pont =  $\lambda$  %ponteiro para o nó procurado – inicia com vazio%
    ptr = ptlista.prox %ponteiro de percurso%
    while ptr  $\neq$   $\lambda$  do
        if ptr.chave < x then %chave buscada é maior%
            ant = ptr
            ptr = ptr.prox
        else if ptr.chave == x then %achou a chave%
            pont = ptr;
            ptr =  $\lambda$  %condição de fim do enquanto%
    return ant, pont
```

Busca em Lista Ordenada,
com Alocação Encadeada,
em Python

```
def busca(chave):  
    global primeiro  
    ponteiros = Ponteiros()  
    ponteiros.pont = primeiro.prox  
    ponteiros.ant = primeiro  
    while ((ponteiros.pont != None) and  
(ponteiros.pont.valor < chave)):  
        ponteiros.ant = ponteiros.pont  
        ponteiros.pont = ponteiros.pont.prox  
    return ponteiros
```

Inserção de um nó em lista encadeada

- Após a busca para localizar a posição onde realizar a inserção, a inclusão ocorre apenas manipulando ponteiros.



Inserção de um nó em lista encadeada

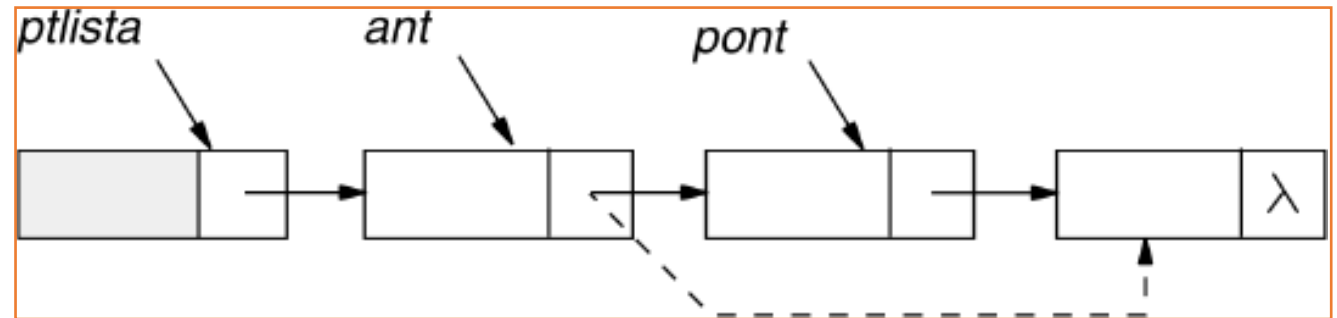
```
function incluir(ptlista, x)
    ant, pont = busca-enc(x, ptlista, ant, pont)
    if pont ==  $\lambda$  then
        new(pt); %criar novo nó%
        pt.chave = x
        pt.prox = ant.prox;
        ant.prox = pt
    else "elemento já existe na tabela"
```

Inclusão em Lista
Ordenada, com Alocação
Encadeada, em Python

```
def incluir(chave):  
    ponteiros = busca(chave)  
    if (ponteiros.pont == None or  
ponteiros.pont.valor > chave):  
        novo = Elemento()  
        novo.valor = chave  
        novo.prox = ponteiros.pont  
        ponteiros.ant.prox = novo  
    else:  
        print("Elemento existe na lista")
```

Remoção de um nó em lista encadeada

- Após localizar o nó a ser excluído, executando a busca, a exclusão ocorre por manipulação de ponteiros.



Remoção de um nó em lista encadeada

```
function excluir(x, ptlista)
    ant, pont = busca-enc(x, ptlista)
    if pont  $\neq \lambda$  then
        ant.prox := pont.prox ;
        free(pont)
    else “nó não se encontra na lista”
```


Exclusão em Lista
Ordenada, com Alocação
Encadeada, em Python

```
def excluir(chave):  
    ponteiros = busca(chave)  
    if (ponteiros.pont != None and  
ponteiros.pont.valor == chave):  
        ptaux = ponteiros.pont  
        ponteiros.ant.prox = ponteiros.pont.prox  
        del ptaux  
    else:  
        print("Chave inexistente")
```

```
typedef struct listano ListaNo;  
struct listano{
```

```
    int chave;  
    char info[15];  
    ListaNo* prox;
```

```
};  
ListaNo* prim;  
ListaNo* ant;
```

```
ListaNo* inicialista(ListaNo* prim)  
{  
    prim = (ListaNo*) malloc(sizeof(ListaNo));  
    prim->prox = NULL;  
    prim->chave = 0;  
    return prim;  
}
```

Implementação de
Lista Encadeada em
C, com nó
cabeçalho.

```
ListaNo* buscalista(ListaNo* prim, int chave1)
{
    ListaNo* pont = NULL;
    ant = prim;
    ListaNo* ptr = prim->prox;
    while ((ptr != NULL) and (ptr->chave < chave1))
    {
        ant = ptr;
        ptr = ptr->prox;
    }
    if ((ptr == NULL) or (ptr->chave != chave1))
        pont = NULL;
    return pont;
}
```

Implementação de
Lista Encadeada em
C, com nó
cabeçalho.

```
void insereno(ListaNo* prim, int chave, char *nome)
{
    ListaNo* pont;
    ListaNo* novo;
    int i;

    pont = buscalista(prim, chave);
    if (pont == NULL)
    {
        novo = (ListaNo*) malloc(sizeof(ListaNo));
        novo->chave = chave;
        for (i=0; i<15; ++i)
            novo->info[i] = nome[i];
        novo->prox = ant->prox;
        ant->prox = novo;
    }
    else
        printf("Elemento já está na tabela.");
}
```

Implementação de
Lista Encadeada em
C, com nó
cabeçalho.

```
void removeno(int chave, char *nome)
{
    ListaNo* pont;
    ListaNo* ptaux;
    int i;

    pont = buscalista(prim, chave);
    if (pont != NULL)
    {
        ptaux = pont;
        ant->prox = pont->prox;
        for (i=0; i<15; ++i)
            nome[i] = ptaux->info[i];
        free(ptaux);
    }
    else
        printf("Chave nao existe na lista.");
}
```

Implementação de
Lista Encadeada em
C, com nó
cabeçalho.